

Monte Carlo Methods

Introduction

Generating Samples

Markov Chain Monte Carlo + Discrepancy

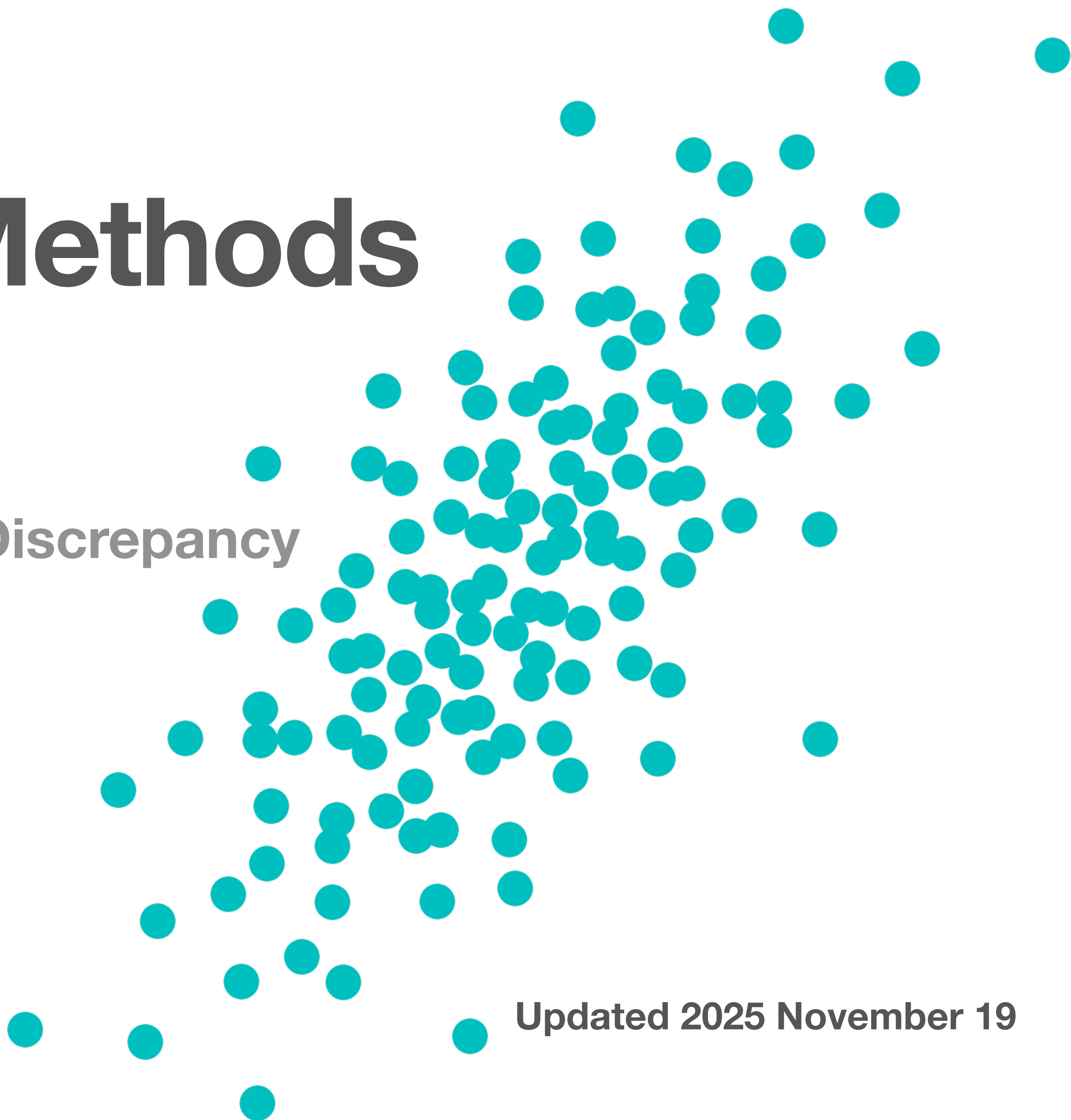
Improving Efficiency

Selected Topics

Git website and repository
Canvas

Fred Hickernell, Fall 2025

Updated 2025 November 19



Selected Topics

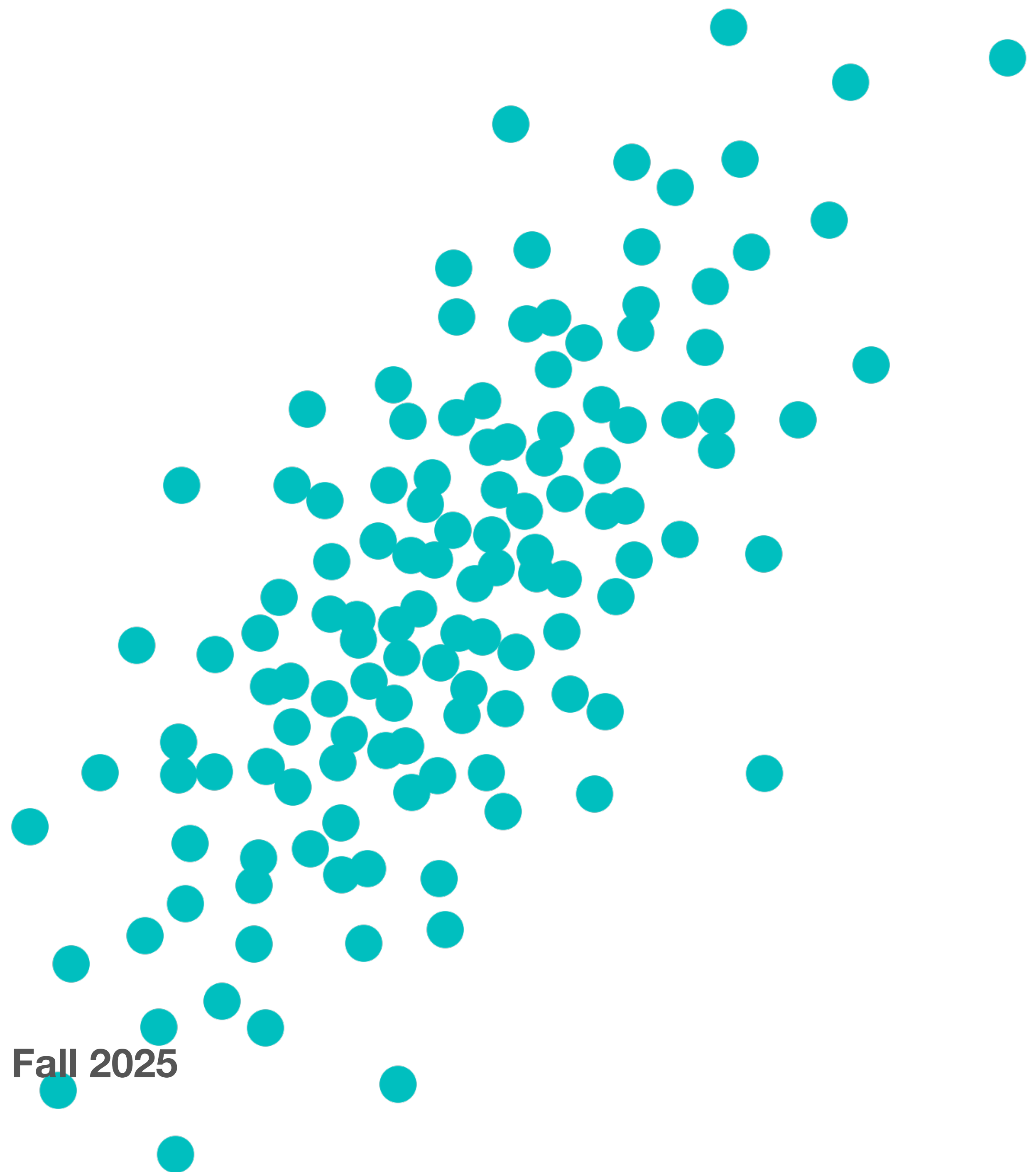
Owen, Chapters ???

Project Presentations Nov 24–25

Take-Home Final Available Dec 3

Take-Home Final Due and
In-Class Final Dec 11, 2–4PM,
WH 115

MATH 565 Monte Carlo Methods, Fred Hickernell, Fall 2025





Parallel Computing

Many personal computers now have GPUs built in. This allows one to perform computations in parallel using PyTorch. Here are some things to keep in mind

- Monte Carlo calculations are **pleasantly parallel** because the function values can be obtained independently, and one can even compute multiple expectations independently.
- Calculations on GPUs are typically using **32-bit floating point**, not 64-bit floating point. This means that we have about 6 significant digits of accuracy, not 15.
- For small problems, GPUs may not be better than CPUs, but for **large numbers of samples** or **large numbers of similar problems**, the speed-up can be impressive.



Gradient Descent

Optimization problems take the form

$$\boldsymbol{\theta}_* = \underset{\boldsymbol{\theta} \in \mathbb{R}^m}{\operatorname{argmin}} \operatorname{Loss}(\boldsymbol{\theta})$$

Gradient descent updates the estimate of $\boldsymbol{\theta}_*$ iteratively by moving **downhill**:

$$\boldsymbol{\theta}_0 \text{ given, } \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \operatorname{Loss}(\boldsymbol{\theta}_k),$$

where η is the step size or learning rate.

- This will often converge since $\nabla \operatorname{Loss}(\boldsymbol{\theta}_k) \rightarrow \mathbf{0}$ as $\boldsymbol{\theta}_k \rightarrow \boldsymbol{\theta}_*$
- But it may converge to a **local minimum**
- This may require more steps than methods that use **second derivatives**, but each step should be cheaper and easier to compute



Stochastic Gradient Descent

Loss function in **regression** often take the form

$$\text{Loss}(\boldsymbol{\theta}) = \sum_{i=1}^N \text{loss}_i(\boldsymbol{\theta}; \text{data}), \quad \text{e.g., } \text{loss}_i(\boldsymbol{\theta}; \text{data}) = [y_i - \theta_1 \exp(\theta_2 x_i)]^2$$

The loss to be optimized is a **sum** of many losses. If N is large, then the gradient is **expensive**. An alternative is **stochastic gradient descent**

$$\boldsymbol{\theta}_0 \text{ given, } \boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \text{loss}_{i_k}(\boldsymbol{\theta}_k)$$

where $i_0, i_1, \dots$ are chosen IID uniform over $1, \dots, N$, which is **cheap**.



Stochastic Gradient Descent

Loss function in **regression** often take the form

$$\text{Loss}(\boldsymbol{\theta}) = \sum_{i=1}^N \text{loss}_i(\boldsymbol{\theta}; \text{data}), \quad \text{e.g., } \text{loss}_i(\boldsymbol{\theta}; \text{data}) = [y_i - \theta_1 \exp(\theta_2 x_i)]^2$$

gradient is **expensive**. An alternative is **stochastic gradient descent**

vanilla $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla \text{loss}_{i_k}(\boldsymbol{\theta}_k)$

decaying η $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \nabla \text{loss}_{i_k}(\boldsymbol{\theta}_k), \quad \eta_k = \frac{\eta_0}{\sqrt{1 + k/100}}$

mini-batch $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \frac{1}{B} \sum_{i_{kj}=1}^B \nabla \text{loss}_{i_{kj}}(\boldsymbol{\theta}_k), \quad i_{kj} \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}\{1, \dots, N\}$

mini-batch + decay $\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta_k \frac{1}{B} \sum_{i_{kj}=1}^B \nabla \text{loss}_{i_{kj}}(\boldsymbol{\theta}_k), \quad i_{kj} \stackrel{\text{i.i.d.}}{\sim} \mathcal{U}\{1, \dots, N\} \quad \eta_k = \frac{\eta_0}{\sqrt{1 + k/100}}$